

Fraud Rule Engine — Architecture Summary

Capitec Bank S3 Submission

2026-05-20

Elevator pitch — 60 seconds

A production-grade fraud rule engine for Capitec Bank. Spring Boot 4 + Java 21. Ingests transaction events (REST + Kafka), evaluates them against versioned YAML-defined rules, and emits explainable decisions on `tx.decisions.v1` with the full matched-rule trace and the rule-set version stamped on every row — so a reviewer can answer “why was this flagged” months later.

Six composed rules ship out of the box: blacklisted merchant, velocity burst, high-amount-on-new-account, cross-border-high-value, off-hours large transaction, new-device high-amount. Stateful predicates (`velocity`, `deviceFingerprintNew`) are backed by Redis so the engine works correctly across multiple API replicas.

The deterministic engine is the **only** source of truth. An optional Ollama AI advisory runs at a separate endpoint with a 2-second timeout and a hard `humanReviewRequired=true` flag — useful for analysts, structurally incapable of overriding a verdict.

Banking-grade defaults throughout: RFC 7807 problem-details on every error, JWT + service-API-key with method-level `@PreAuthorize`, Bucket4j + Redis rate limit per JWT subject, Idempotency-Key with SHA-256 body hash + subject binding, transactional outbox with Postgres `SELECT FOR UPDATE SKIP LOCKED` for multi-replica safety, audit log via async `@EventListener`, structured-JSON logs with `PiiRedactor` for POPIA. ArchUnit pins the `api does not depend on persistence` boundary.

98 % engine line coverage. 76 tests (unit + integration + ArchUnit + Testcontainers). Docker Compose runs the whole stack (Postgres + Redis + Redpanda + API) in three commands. Submission ZIP under 50 MB. CI green on GitHub Actions: `./mvnw verify` + gitleaks scan + Jacoco gate, every push.